PCM ENCODING AND DECODING OF MUSIC

BIT WISE ANALYSIS USING 4,8, AND 12-BIT DEPTH

Course Based project - ADC

Submitted by

VALLAPUREDDY MALLA REDDY 1602-23-735-150

RAJESHWARI KUSHALA 1602-23-735-164

MORISETTY SRINIVAS 1602-23-735-182

Department of Electronics & Communication Engineering

VASAVI COLLEGE OF ENGINEERING (AUTONOMOUS)

IBRAHIMBAGH, HYDERABAD-500031

November 2025

Aim of the Project :

The aim of this project is to analyze the impact of quantization bit depth on the quality, accuracy, and storage efficiency of PCM (Pulse Code Modulation) encoded audio signals. This involves implementing PCM encoding and decoding at 4-bit, 8-bit, and 12-bit resolutions, comparing the resulting audio waveforms, quantization noise levels, and perceptual sound quality.

Objectives :

1.To implement PCM encoding and decoding algorithms for audio signals using **4-bit, 8-bit, and 12-bit quantization levels**.

2.To analyze the effect of different bit depths on audio quality, including waveform distortion, quantization noise, and dynamic range.

3.To compare the reconstructed audio signals for each bit depth and evaluate perceptual and measurable differences.

4.To calculate and compare signal-to-quantization-noise ratio (SQNR) for 4-, 8-, and 12-bit PCM systems.

INTRODUCTION:

The evolution of modern communication and multimedia systems has increased the demand for accurate and efficient audio storage and transmission. Traditionally, audio signals exist in analog form, where the signal varies continuously with time. Although analog audio offers a natural representation of sound, it is prone to noise, distortion, and quality degradation during storage or transmission. These limitations have led to the widespread adoption of digital audio techniques, particularly in applications such as telecommunication, music production, audio streaming platforms, and compact disc (CD) recording.

To enable digital processing, the analog audio signal must be converted into a digital format. This process involves two fundamental operations: sampling and quantization. Sampling refers to measuring the amplitude of the audio signal at uniform time intervals. The rate at which these samples are captured is called the sampling frequency. According to the Nyquist Sampling Theorem, the sampling frequency must be at least twice the maximum frequency present in the analog signal to ensure accurate reconstruction during playback.

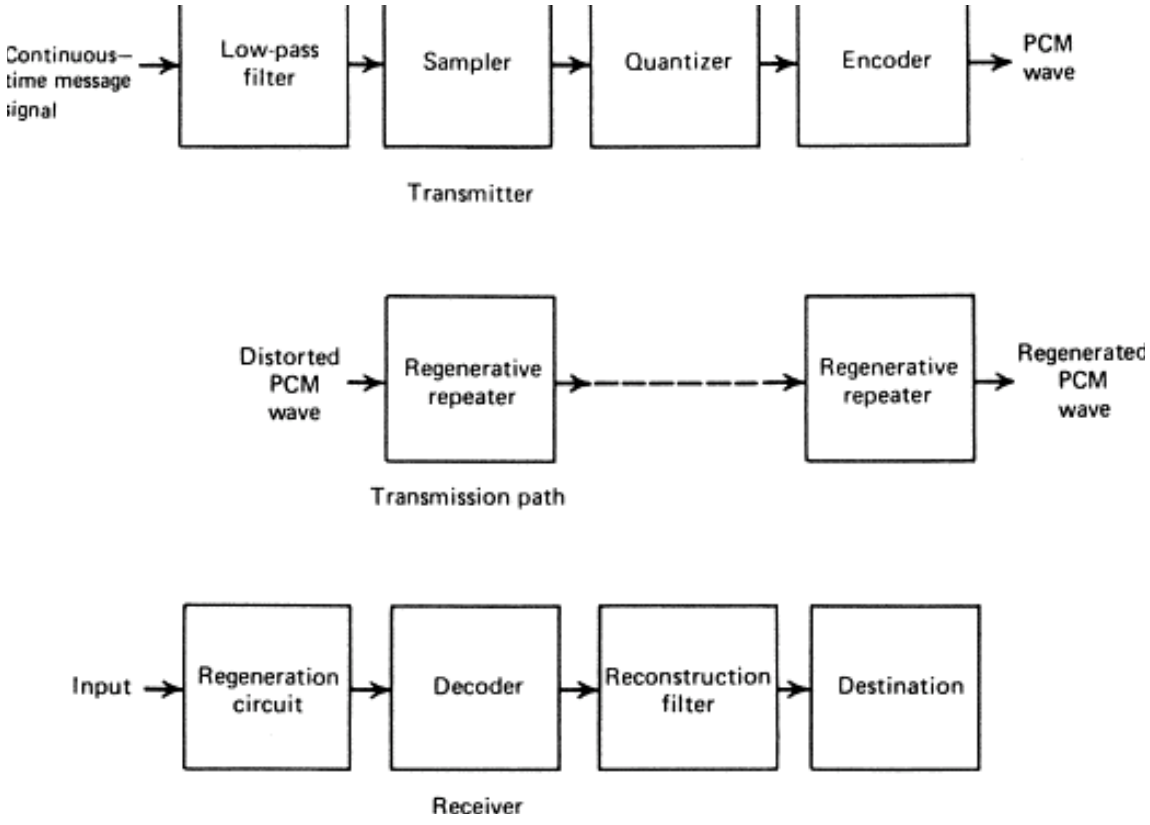
Once the signal is sampled, the amplitude values are mapped to discrete numerical levels through quantization. The number of levels available during this step depends on the bit depth, which determines the resolution of the digital representation. Higher bit depth provides more quantization levels, resulting in improved dynamic range and reduced quantization noise. For instance, 4-bit PCM yields only 16 discrete levels and introduces audible distortion, whereas 12-bit PCM provides 4096 levels, significantly improving audio fidelity.

Finally, the quantized values are encoded into binary format to generate a digital bit stream through Pulse Code Modulation (PCM). PCM is one of the most widely used digital audio encoding techniques because it preserves the original waveform characteristics without compression or loss of information. Due to its simplicity and accuracy, PCM is used extensively in digital telephony, audio recording systems, and sound processing applications.

METHODOLOGY:

The methodology for the PCM (Pulse Code Modulation) encoding and decoding experiment follows a structured process for converting an analog audio signal into digital form and reconstructing it back to analog. The procedure begins with analog signal acquisition, where a continuous-time audio signal is recorded or taken as input. Next, the signal undergoes sampling, in which it is measured at a rate higher than twice its highest frequency (Nyquist rate) to avoid aliasing. After sampling, quantization is performed to map each sample to the nearest level within a finite set of amplitude values, a step that introduces quantization error but improves with higher bit depth. Finally, encoding converts these quantized values into binary code, producing a digital PCM stream suitable for storage, processing, or transmission.

Steps for PCM Decoding involve reversing the encoding process to recover the original analog signal. First, binary-to-decimal conversion is performed, where the received binary codes are translated back into numerical values representing the quantized levels. Next, reconstruction is carried out by simulating a Digital-to-Analog Converter (DAC), which converts these discrete values into a continuous-time signal. Finally, an optional low-pass filtering stage is applied to smooth the output and reduce the staircase appearance caused by quantization, producing a cleaner reconstructed analog waveform.



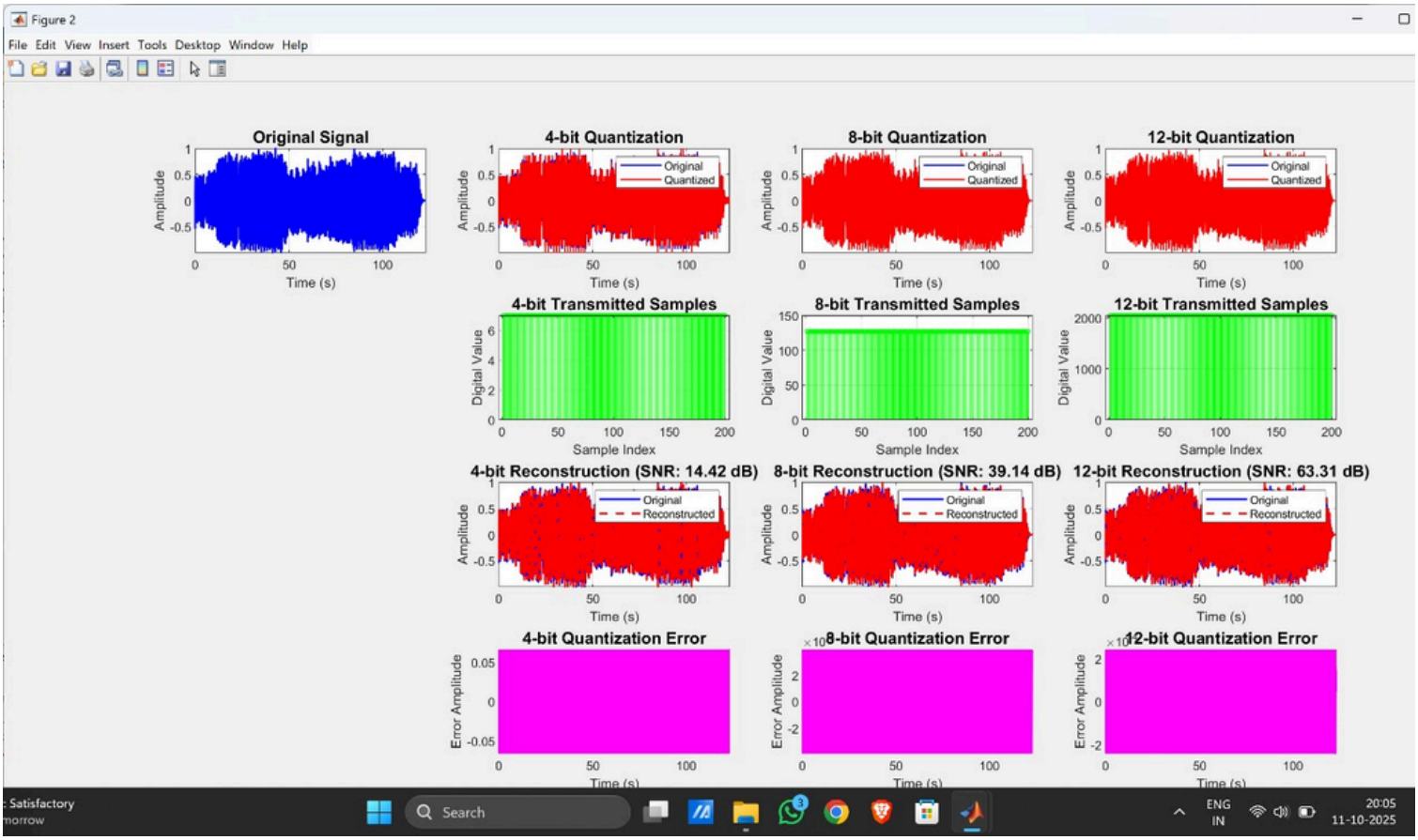
SOFTWARE IMPLEMENTATION:

The software implementation of the PCM encoding and decoding system was carried out using MATLAB, which provides efficient tools for audio processing, numerical computation, and visualization. The input music signal was first imported using built-in functions and normalized to ensure it remained within the standard amplitude range for quantization. For each selected bit depth (4, 8, and 12 bits), the program calculated the corresponding number of quantization levels and step size. Uniform quantization was applied by mapping the continuous audio samples to their nearest discrete levels, generating digital PCM samples that represent the encoded audio.

In the next phase, the quantized samples were encoded into binary sequences using MATLAB's integer-to-binary conversion functions, effectively simulating the PCM transmission process. These binary codes were then decoded back into digital values to reconstruct the audio signal. The reconstructed waveform was scaled back to its original amplitude range, allowing direct comparison with the original signal. MATLAB's signal processing functions were used to evaluate the accuracy of the reconstruction by computing performance metrics such as Signal-to-Noise Ratio (SNR), Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and Total Harmonic Distortion (THD).

Finally, a comprehensive set of plots was generated to visually analyze the impact of different bit depths on quantization accuracy and audio reconstruction quality. These visualizations included the original waveform, quantized waveforms for each bit depth, transmitted digital samples, reconstructed signals, and quantization error plots. The software also produced a summary table displaying the numerical values of SNR, MSE, PSNR, and THD for 4-, 8-, and 12-bit systems. Overall, the software implementation successfully demonstrated how increasing bit depth improves PCM audio fidelity while reducing quantization noise and signal distortion.

RESULTS AND DISCUSSION:



The results clearly demonstrate the influence of quantization bit depth on the quality of PCM-encoded audio signals. The original waveform appears smooth and continuous, while the 4-bit, 8-bit, and 12-bit quantized signals show progressively finer amplitude resolution as the number of quantization levels increases. In the 4-bit case, the waveform is heavily distorted due to coarse quantization steps, resulting in a low SNR of approximately 14.42 dB. This indicates significant quantization noise, which is also observable in the reconstructed signal and its corresponding error plot. With 8-bit quantization, the reconstruction quality improves noticeably, and the SNR increases to around 39.14 dB, showing that the quantized samples more closely follow the original waveform. The 12-bit quantization produces the highest fidelity, with an SNR of about 63.31 dB, and the error signal becomes extremely small, indicating minimal loss of information during encoding and decoding. The transmitted sample plots further confirm that higher bit depths produce a greater number of discrete amplitude levels, allowing more accurate digital representation of the audio. Overall, the results validate that increasing bit depth significantly reduces quantization noise, enhances reconstruction accuracy, and improves the perceptual and measurable quality of the PCM audio signal.

Bits	Levels	Step_Size	SNR_dB	MSE	PSNR_dB	THD_percent
4	16	0.13332	14.423	0.0015385	28.129	19.004
8	256	0.0078421	39.136	5.1977e-06	52.842	1.1046
12	4096	0.00048834	63.313	1.9864e-08	77.019	0.068286

The numerical results clearly show how increasing quantization bit depth improves the fidelity of the PCM-encoded audio signal. At 4 bits, the system has only 16 quantization levels, resulting in a large step size of 0.13332, which introduces significant quantization error. This is reflected in the low SNR of 14.423 dB, high MSE of 0.0015385, and relatively low PSNR of 28.129 dB. The THD (19.004%) is also very high, indicating strong harmonic distortion caused by coarse quantization. When the bit depth increases to 8 bits (256 levels), the step size reduces drastically to 0.0078421, which improves the accuracy of amplitude representation. Correspondingly, the SNR increases to 39.136 dB, the MSE drops to 5.1977×10^{-6}, and PSNR rises to 52.842 dB, showing a substantial improvement in signal quality. The THD also decreases dramatically to 1.1046%, meaning the reconstructed audio is much closer to the original with far fewer distortion components. At 12 bits, the system achieves 4096 levels with an extremely fine step size of 0.00048834, producing near-transparent audio quality. The SNR reaches 63.313 dB, MSE falls to 1.9864×10^{-8}, and PSNR peaks at 77.019 dB, indicating very minimal quantization noise. The THD drops to only 0.068286%, showing that harmonic distortion becomes almost negligible. Overall, these results confirm that higher bit depths significantly enhance audio quality by reducing quantization noise, minimizing distortion, and improving both objective metrics and perceptual clarity.

APPENDIX:

```
function pcm_music_analysis()
    % PCM Encoding & Decoding of Music Signal Analysis
    % This function performs quantization analysis on user's audio file

    clc;clear;close all;

    % Get user input for audio file
    [filename, pathname] = uigetfile({'*.wav;*.mp3;*.m4a', 'Audio Files (*.wav, *.mp3, *.m4a)'}, ...
        'Select an audio file');

    if isequal(filename, 0)
        disp('No file selected. Using default sample.');
        % Create a sample signal if no file is selected
        fs = 8000; % Sampling frequency
        t = 0:1/fs:2; % 2 seconds
        original_signal = sin(2*pi*440*t) + 0.5*sin(2*pi*880*t); % A4 + A5 notes
    else
        % Read the audio file
        filepath = fullfile(pathname, filename);
        [original_signal, fs] = audioread(filepath);

        % Convert to mono if stereo
        if size(original_signal, 2) > 1
```

```
    original_signal = mean(original_signal, 2);
  end

  % Use the full signal (no truncation)
  original_signal = original_signal(:);

  % Normalize signal
  original_signal = original_signal / max(abs(original_signal));
end

% Time vector
t = (0:length(original_signal)-1) / fs;

% Standalone figure for Original Signal
figure('Position', [200, 200, 1200, 400]);
plot(t, original_signal, 'b', 'LineWidth', 1.2);
title('Original Audio Signal');
xlabel('Time (s)'); ylabel('Amplitude');
grid on;

% Quantization levels
bit_levels = [4, 8, 12];

% Initialize results storage
results = struct();
```

```
% Create figure for summary plots
figure('Position', [100, 100, 1400, 1000]);

% Plot 1: Original Signal (in summary figure)
subplot(4, 4, 1);
plot(t, original_signal, 'b', 'LineWidth', 1.5);
title('Original Signal', 'FontSize', 12, 'FontWeight', 'bold');
xlabel('Time (s)'); ylabel('Amplitude');
grid on; axis tight;

% Initialize table data
table_data = [];

% Loop over different bit depths
for i = 1:length(bit_levels)
    bits = bit_levels(i);

    % PCM Encoding & Decoding
    [quantized_signal, reconstructed_signal, quantization_levels, step_size, ...
     transmitted_samples, snr_db, mse, psnr_db, thd_percent] = ...
     pcm_encode_decode(original_signal, bits);

    % Store results
    results.(sprintf('bits_%d', bits)).quantized = quantized_signal;
```

```
results.(sprintf('bits_%d', bits)).reconstructed = reconstructed_signal;
results.(sprintf('bits_%d', bits)).snr = snr_db;
results.(sprintf('bits_%d', bits)).mse = mse;
results.(sprintf('bits_%d', bits)).psnr = psnr_db;
results.(sprintf('bits_%d', bits)).thd = thd_percent;

% Add to table data
table_data = [table_data; bits, quantization_levels, step_size, ...
        snr_db, mse, psnr_db, thd_percent];

% === Print results to Command Window ===
fprintf('\n=== %d-bit Quantization Results ===\n', bits);
fprintf('Quantization Levels : %d\n', quantization_levels);
fprintf('Step Size         : %.6f\n', step_size);
fprintf('SNR (dB)          : %.4f\n', snr_db);
fprintf('MSE               : %.6e\n', mse);
fprintf('PSNR (dB)         : %.4f\n', psnr_db);
fprintf('THD (%%)           : %.4f\n', thd_percent);
fprintf('----------------------------------\n');

% Plot quantized vs original
subplot(4, 4, 1+i);
plot(t, original_signal, 'b-', 'LineWidth', 1, 'DisplayName', 'Original');
hold on;
plot(t, quantized_signal, 'r-', 'LineWidth', 1, 'DisplayName', 'Quantized');
```

```
title(sprintf('%d-bit Quantization', bits), 'FontSize', 12, 'FontWeight', 'bold');
xlabel('Time (s)'); ylabel('Amplitude'); legend('show');
grid on; axis tight;

% Plot transmitted samples (first 200 samples for clarity)
subplot(4, 4, 5+i);
n_samples = min(200, length(transmitted_samples));
stem(1:n_samples, transmitted_samples(1:n_samples), 'g', 'MarkerSize', 3);
title(sprintf('%d-bit Transmitted Samples', bits), 'FontSize', 12, 'FontWeight', 'bold');
xlabel('Sample Index'); ylabel('Digital Value');
grid on;

% Plot reconstruction comparison
subplot(4, 4, 9+i);
plot(t, original_signal, 'b-', 'LineWidth', 1.5, 'DisplayName', 'Original');
hold on;
plot(t, reconstructed_signal, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Reconstructed');
title(sprintf('%d-bit Reconstruction (SNR: %.2f dB)', bits, snr_db), ...
    'FontSize', 12, 'FontWeight', 'bold');
xlabel('Time (s)'); ylabel('Amplitude'); legend('show');
grid on; axis tight;

% Plot quantization error
subplot(4, 4, 13+i);
error_signal = original_signal - reconstructed_signal;
```

```
    plot(t, error_signal, 'm', 'LineWidth', 1);
    title(sprintf('%d-bit Quantization Error', bits), 'FontSize', 12, 'FontWeight', 'bold');
    xlabel('Time (s)'); ylabel('Error Amplitude');
    grid on; axis tight;

    % Save reconstructed audio file
    if ~isequal(filename, 0)
      [~, name, ~] = fileparts(filename);
      output_filename = sprintf('%s_%dbit_reconstructed.wav', name, bits);
      audiowrite(output_filename, reconstructed_signal, fs);
      fprintf('Saved: %s\n', output_filename);
    end
  end

  % Detailed figures for each bit level
  plot_detailed_results(4, t, original_signal, results);
  plot_detailed_results(8, t, original_signal, results);
  plot_detailed_results(12, t, original_signal, results);

  % === Print summary table at the end ===
  fprintf('\n=== Summary Table ===\n');
  Summary = array2table(table_data, ...
    'VariableNames', {'Bits', 'Levels', 'Step_Size', 'SNR_dB', 'MSE', 'PSNR_dB',
'THD_percent'});
  disp(Summary);
end
```

```
% ------------------------------------------------
% Helper function for detailed figures per bit level
% ------------------------------------------------
function plot_detailed_results(bits, t, original_signal, results)
    figure('Position', [200, 200, 1200, 800]);

    % Original vs Reconstructed
    subplot(2,2,1);
    plot(t, original_signal, 'b', 'LineWidth', 1.2); hold on;
    plot(t, results.(sprintf('bits_%d', bits)).reconstructed, 'r--', 'LineWidth', 1.2);
    title(sprintf('Original vs %d-bit Reconstructed', bits));
    legend('Original', sprintf('%d-bit', bits));
    xlabel('Time (s)'); ylabel('Amplitude'); grid on;

    % Transmitted Samples (first 200)
    subplot(2,2,2);
    stem(results.(sprintf('bits_%d', bits)).quantized(1:200), 'g', 'MarkerSize', 3);
    title(sprintf('%d-bit Transmitted Samples (First 200)', bits));
    xlabel('Sample Index'); ylabel('Quantized Value'); grid on;

    % Error waveform
    subplot(2,2,3);
    plot(t, original_signal - results.(sprintf('bits_%d', bits)).reconstructed, 'm');
    title(sprintf('%d-bit Quantization Error', bits));
```

```
    xlabel('Time (s)'); ylabel('Error Amplitude'); grid on;

    % Error histogram
    subplot(2,2,4);
    histogram(original_signal - results.(sprintf('bits_%d', bits)).reconstructed, 50, 'FaceColor','c');
    title(sprintf('Histogram of %d-bit Quantization Error', bits));
    xlabel('Error Amplitude'); ylabel('Count'); grid on;
end

% ------------------------------------------------
% PCM Encode/Decode function
% ------------------------------------------------
function [quantized_signal, reconstructed_signal, quantization_levels, step_size, ...
        transmitted_samples, snr_db, mse, psnr_db, thd_percent] = pcm_encode_decode(signal, bits)

    % Number of quantization levels
    quantization_levels = 2^bits;

    % Find signal range
    signal_min = min(signal);
    signal_max = max(signal);
    signal_range = signal_max - signal_min;

    % Step size
```

```
step_size = signal_range / (quantization_levels - 1);

% Quantization (Encoding)
normalized_signal = (signal - signal_min) / signal_range * (quantization_levels - 1);
quantized_indices = round(normalized_signal);
quantized_indices = max(0, min(quantization_levels - 1, quantized_indices));

% Quantized signal
quantized_signal = quantized_indices / (quantization_levels - 1) * signal_range +
signal_min;

% Transmitted samples (digital values)
transmitted_samples = quantized_indices;

% Reconstruction
reconstructed_signal = quantized_signal;

% Metrics
signal_power = mean(signal.^2);
noise_power = mean((signal - reconstructed_signal).^2);
if noise_power == 0
   snr_db = Inf;
else
   snr_db = 10 * log10(signal_power / noise_power);
end
```

```
    mse = mean((signal - reconstructed_signal).^2);
    max_signal = max(abs(signal));
    if mse == 0
        psnr_db = Inf;
    else
        psnr_db = 20 * log10(max_signal) - 10 * log10(mse);
    end

    fundamental_power = signal_power;
    distortion_power = noise_power;
    if fundamental_power == 0
        thd_percent = 0;
    else
        thd_percent = 100 * sqrt(distortion_power / fundamental_power);
    end
end
```